

Course Name: operating systems

Date: 10/8/26

Name: Rithika P

Reg.No: 192565002

Course Code: CSA0406

## C03-AT2

1) Embedded System - static linking Vs Dynamic Linking.

a) Which is more suitable?

Dynamic linking is more suitable for this embedded system. The system has only 64MB physical memory and runs several small applications. In static linking, all required modules are included in the executable before execution. This makes the executable larger. In dynamic linking, modules or libraries are loaded only when they are needed during execution.

b) Memory & storage requirement

| Aspect          | Static linking              |
|-----------------|-----------------------------|
| Executable size | Large                       |
| Storage space   | More                        |
| Memory usage    | Higher                      |
| Library sharing | Less                        |
| Loading         | All linked modules included |

example:

Suppose 4 applications use the same 2MB library.

$$4 \times 2\text{MB} = 8\text{MB}$$

c)

One disadvantage

The main disadvantage is runtime dependency. There can also be small delay when a module is loaded during execution.

d)

Final Justification

Dynamic linking is the better choice because the system has limited 64 MB memory or runs several applications.

2)

Cloud Server - Contiguous Allocation Vs paging

a)

Evaluation

In contiguous memory allocation, each process must be stored in one continuous block of physical memory.

For eg.

Process A  $\rightarrow$  [continuous memory block]

If many processes start or terminate free memory becomes divided into small holes. This causes external fragmentation.

In paging, physical memory is divided into fixed size frames or processes are divided into fixed-size pages.

A process's pages can be placed in different frames therefore paging is more suitable for a cloud server running hundreds of processes.



Which provides better memory utilization?

Paging provides better memory utilization.

This is because a process does not need one large continuous block of memory.

For eg: Process needs 4 pages. It can be stored as

Page 1 → Frame 2

Page 2 → Frame 8

Page 3 → Frame 15

Page 4 → Frame 20

c) Handling non-contiguous memory

Paging uses a page table to maintain the mapping between logical pages and physical frames.

eg:

| Page   | Frame    |
|--------|----------|
| page 0 | Frame 5  |
| page 1 | Frame 12 |
| page 2 | Frame 3  |
| page 3 | Frame 18 |

Thus, a process can occupy non-contiguous memory.

d) Final Justification

Paging is the better choice because the cloud server has hundreds of processes with different memory requirements. It

- Reduces external fragmentation
- Uses physical memory efficiently
- Supports virtual memory.

3)

## Application - paging Vs Segmentation

The application contains:

- Code
- Global data
- Stack
- Heap
- shared libraries.

a) Which is more suitable?

Segmentation is more suitable for this application. It divides according to its logical parts.

| Segment         | Contains                           |
|-----------------|------------------------------------|
| Code Segment    | Program instructions               |
| Data segment    | Global Variables                   |
| Stack Segment   | Function calls and Local Variables |
| Heap Segment    | Dynamically allocated memory       |
| Library Segment | shared library code                |

b) Protection and Controlled access

Each segment can have different access permission:

For eg.

Code → Read + Execute

Global data → Read + Write

Stack → Read + Write

Heap → Read + Write



If a program tries to write into the code segment, the operating system can detect the invalid access and generate a protection error.

c) One limitation

The major limitation is external fragmentation. Since segments have different sizes. Difficult to find a sufficiently large continuous block for a new segment.

d) Final Justification

Segmentation is suitable because the application consist of clearly defined logical components. It provides

- Logical organization
- Different access permissions
- Memory protection.

4) Modern operating system

a) compare paging, Segmentation or Segmentation with paging.

| Feature            | Paging                    | Segmentation           | Segmentation + paging       |
|--------------------|---------------------------|------------------------|-----------------------------|
| Division           | Fixed-Size Pages          | Variable-size segments | Segments divided into pages |
| Fragmentation      | No external fragmentation | External fragmentation | Very low                    |
| Protection         | Good                      | Very good              | Very good                   |
| Memory utilization | High                      | Moderate               | High                        |

b)

Best Mechanism

Segmentation with paging is the best choice.

It combines the advantages of both techniques.

- Segmentation provides logical organization and protection.
- Paging provides efficient memory utilization.
- It reduces external fragmentation.

c)

How does it handle memory?

A program is first divided into segments such as code, data and stack. Each segment is then divided into pages.

Logical address → Segment → Page → offset → physical frame.

d)

Why are the other two less suitable?

Paging: Good memory utilization, but it does not provide as clear a logical organization of code, data or stack.

Segmentation: provides good protection and logical organization, but can cause external fragmentation.

Segmentation with paging because it provides a good combination of protection, flexibility and memory utilization.



System A and System B

Suitability of each technique

contiguous Allocation:

Simple and Fast, but causes external fragmentation and requires continuous memory

Paging:

uses memory efficiently and allows processes to use non-contiguous memory.

segmentation:

Provides good logical organization and protection but can cause external fragmentation

b) Best Mechanism.

| System                       | Best choice |
|------------------------------|-------------|
| System A → Gaming computer   | Paging      |
| System B → Multi user server | Paging      |

c) Comparison

| Factor              | contiguous   | Paging   | segmentation |
|---------------------|--------------|----------|--------------|
| Fragmentation       | High         | Very Low | High         |
| Memory Utilization  | Low          | High     | Moderate     |
| Protection          | Low          | Good     | Very Good    |
| Address translation | Low overhead | Moderate | Moderate     |
| Flexibility         | Low          | High     | High         |

d)

### Final recommendation

System A - Gaming Computer

→ Paging is recommended because games can be very large. Paging allows the program to run in different physical memory locations and support Virtual memory.

System B - Multi user Server.

Paging is recommended because many processes have different memory requirements. It reduces external fragmentation and uses available memory efficiently.

System A → Paging

System B → Paging

→ It provides high memory utilization, flexibility, process protection, and support for large programs.